



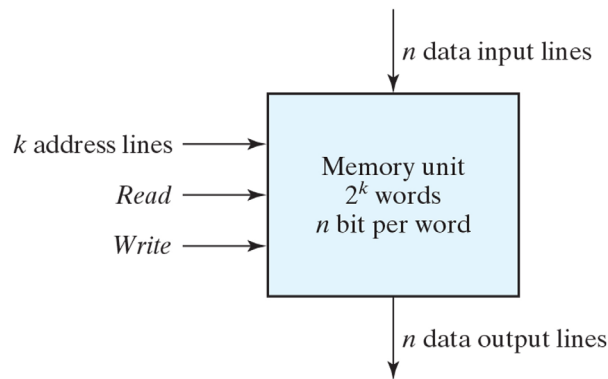
디지털공학

8 주차

7 메모리 및 프로그래머블 논리

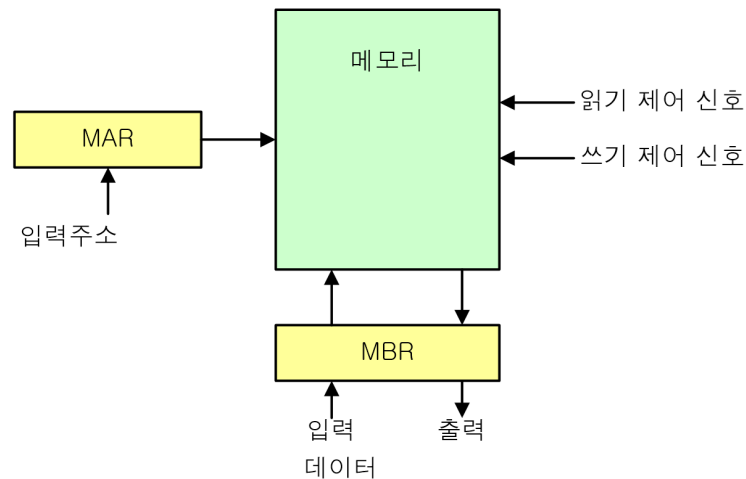
❖ 메모리 구조

- 데이터를 저장하는 셀, 장치와 정보를 주고 받기 위한 주변회로로 구성되어 있음
- 메모리 유닛은 워드 단위로 데이터 처리
- 워드(Word) : 한 번에 읽기/쓰기 처리 되는 비트 수를 칭하며, 대부분의 컴퓨터 메모리는 8비트(1Byte)의 배수를 워드로 사용



Memory address		Memory content
Binary	Decimal	
0000000000	0	1011010101011101
0000000001	1	1010101110001001
0000000010	2	0000110101000110
	⋮	⋮
1111111101	1021	1001110100010100
1111111110	1022	0000110100011110
1111111111	1023	1101111000100101

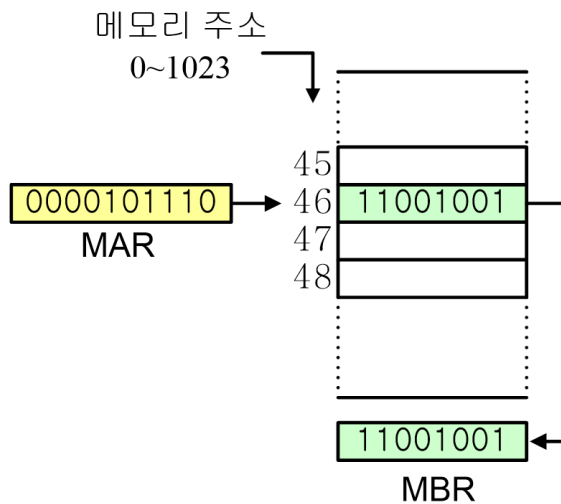
- 메모리는 메모리 주소 레지스터(MAR, Memory Address Register), 메모리 버퍼 레지스터(MBR, Memory Buffer Register), 메모리로 구성되어 있음
- MAR : 메모리 접근 시 접근하고자 하는 영역의 주소 지정
- MBR : 메모리에 읽기/쓰기 동작 처리를 위한 데이터 임시 저장 장소



❖ 메모리 읽기/쓰기 동작

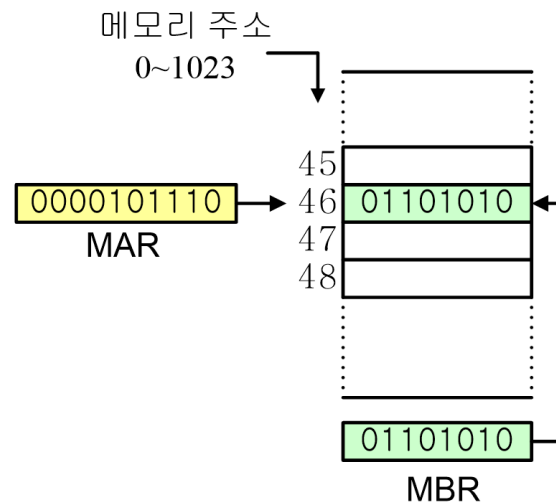
[읽기]

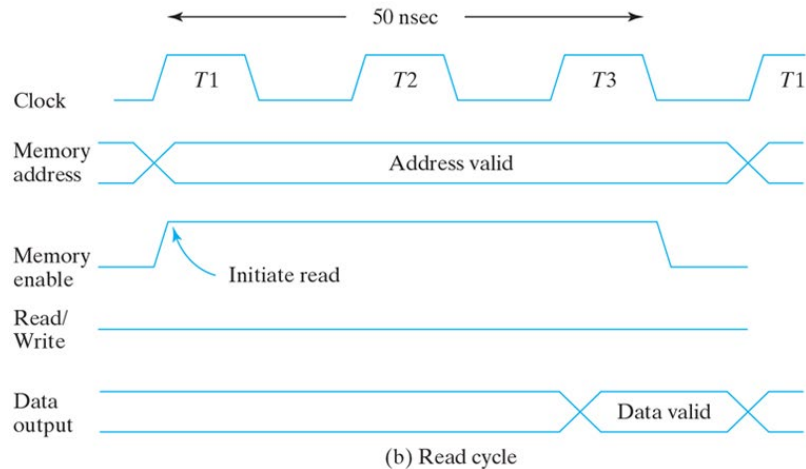
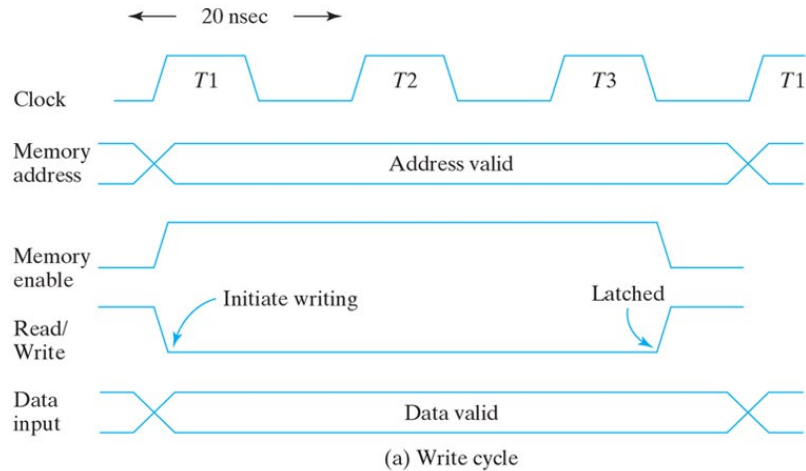
- 메모리 주소를 MAR로 전송
- 읽기 제어신호
- MBR에 저장된 데이터 사용



[쓰기]

- 메모리 주소를 MAR로 전송
- 데이터를 MBR로 전송
- 쓰기 제어신호





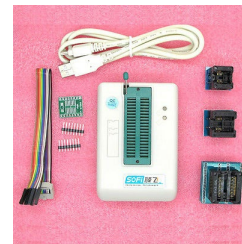
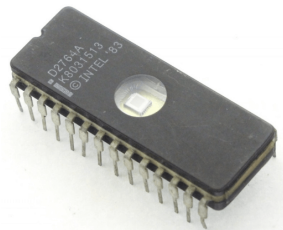
Memory Enable	Read/Write	Memory Operation
0	X	None
1	0	Write to selected word
1	1	Read from selected word

❖ 접근 방법에 의한 분류

- RAM(Random Access Memory) : 접근 시간이 메모리의 어느 위치나 동일
- SAM(Sequential Access Memory) : 순차적 접근에 의해 원하는 위치에 따라 접근 시간이 다름

❖ 기능에 의한 분류

- RWM(Read and Write Memory) : 읽기/쓰기 모두 수행 가능한 메모리로 RAM는 RWM 메모리
- ROM(Read Only Memory) : 읽기만 가능한 메모리를 칭하며, 특정 조건에서만 쓰기가 가능
 - Mask ROM : 제조 공정에서 쓰기 가능
 - Fuse-Link PROM : 한번만 기록이 가능
 - PROM(Programmable ROM) : 사용자에게 의해 한번만 쓰기가 가능
 - EPROM(Erasable PROM) : 자외선으로 저장 내용을 지운 후 다시 쓰기 가능
 - EEPROM(Electrically Erasable PROM) : 전기적으로 내용을 지운 후 다시 쓰기 가능



❖ 저장 방식에 의한 분류

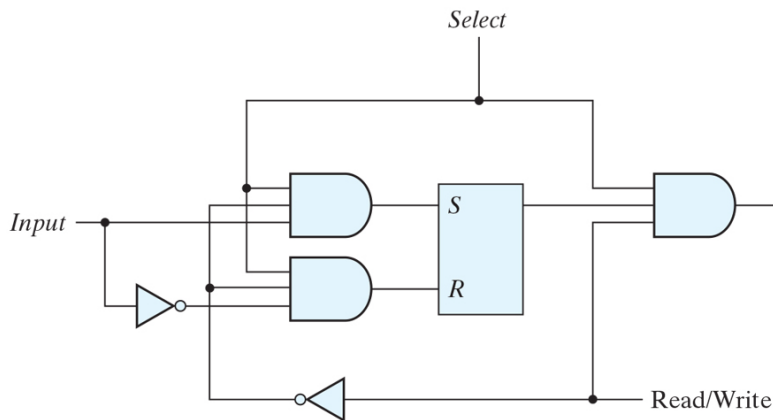
- SRAM(Static RAM) : 2진 정보를 저장할 수 있는 내부 Flip-Flop으로 구성되며, 전원이 공급되는 동안 데이터가 보존. 사용하기 쉽고 읽기/쓰기 Cycle이 짧음
- DRAM(Dynamic RAM) : 2진 정보를 전하가 충전된 형태로 저장. SRAM에 비해 전력소비가 적고 단위 면적 당 더 많은 데이터를 저장할 수 있으나, 충전상태 유지를 위한 Refresh 회로 필요

❖ 데이터 유지성에 의한 분류

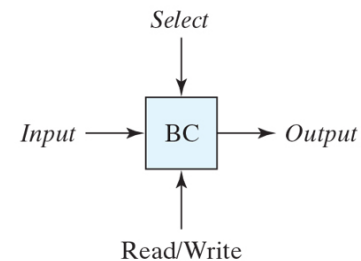
- 휘발성 메모리 : 일정 시간이 지나거나 전원 공급이 중단되면 기록된 내용이 지워지는 메모리로 대표적인 예는 RAM
- 비휘발성 메모리 : 전원 공급이 중단되더라도 기록된 정보가 유지되는 메모리로 Flash Memory, 자기 디스크 등이 있음

❖ SRAM 메모리 셀 구조와 동작

- 아래의 그림은 1비트 정보를 저장할 수 있는 SRAM의 2진 셀 논리회로
- 입력 비트, Read/Write 신호, Select 신호로 구성된 입력과 출력 비트로 구성
- BC(Binary Cell)를 기본단위로 시스템 워드에 맞게 그룹화 하여 메모리 구성



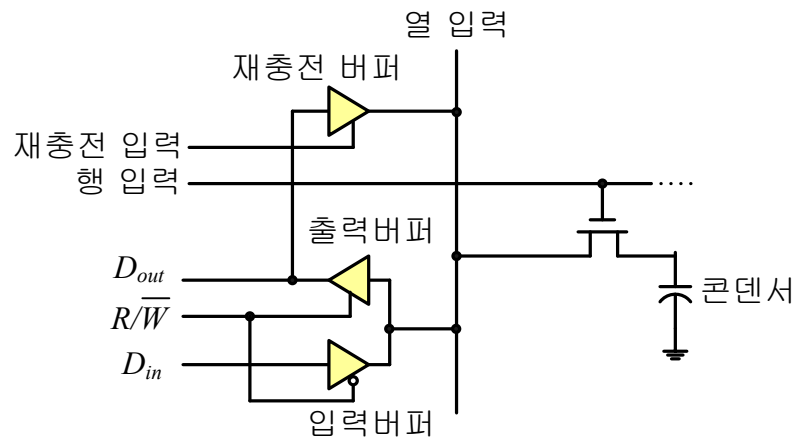
(a) Logic diagram



(b) Block diagram

❖ DRAM 메모리 셀 구조와 동작

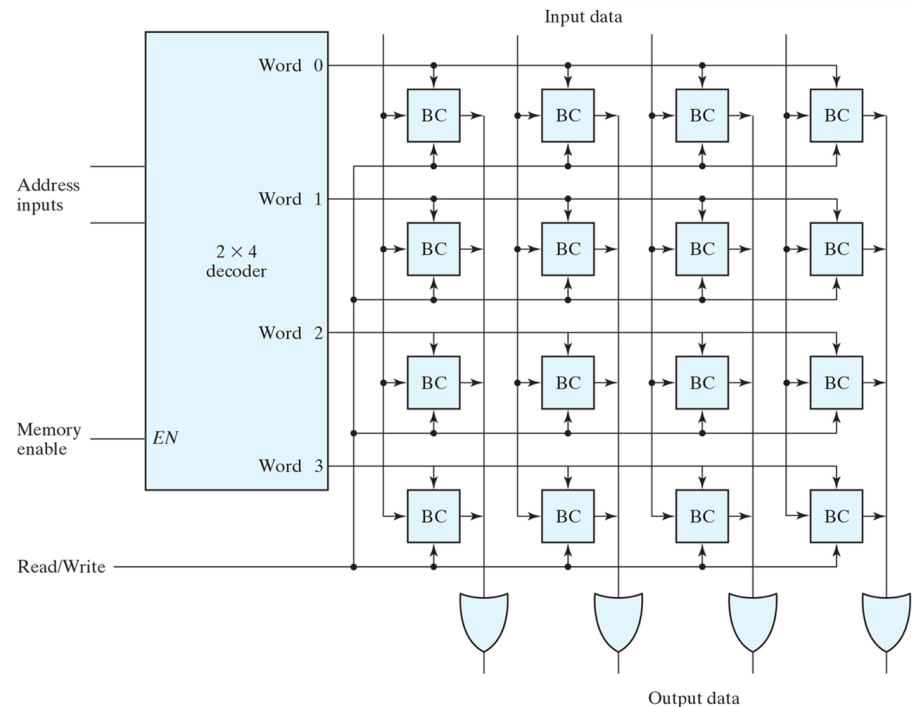
- Capacitor에 전하를 저장하는 형태로 이진 정보 저장
- Capacitor에 저장되어 있는 전하는 일정 시간이 지나면 방전되어 정보 손실 발생
 - ✓ 주기적인 재충전 필요(refreshing)
 - ✓ 재충전을 위한 추가 회로 필요



❖ 메모리 디코딩

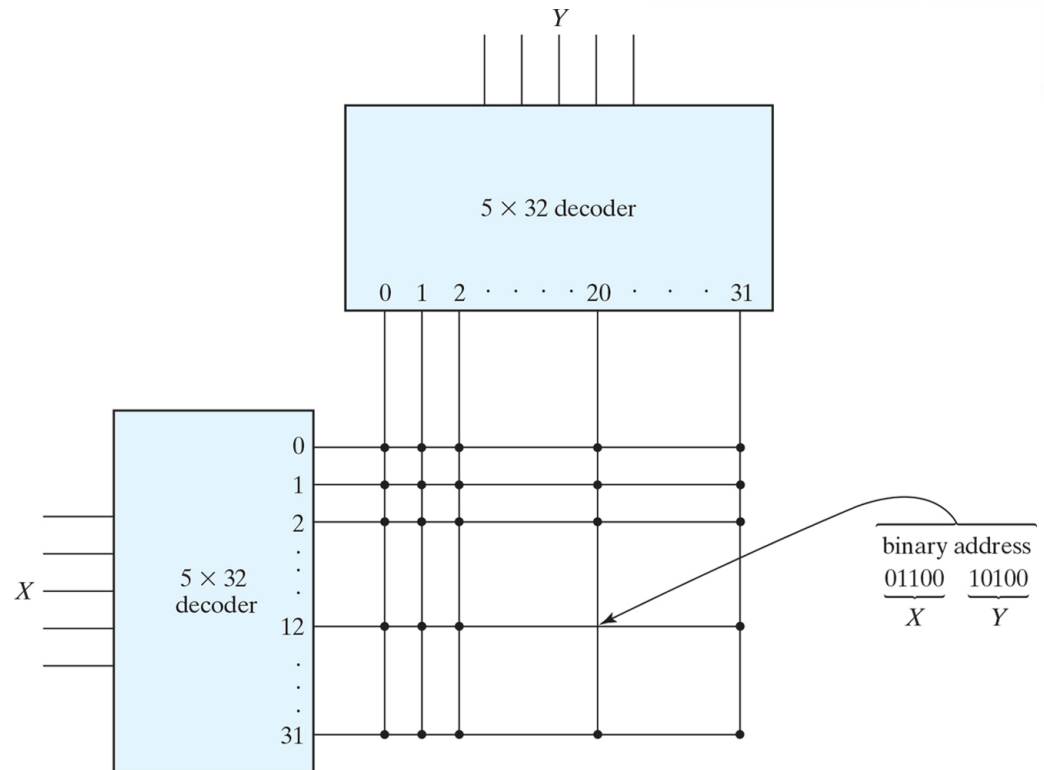
- Memory Enable = 0 : 디코더 출력은 0이고 메모리 선택되지 않음
- Memory Enable = 1 : Address input에 의해 Word가 선택되고 Read/Write 신호에 의해 동작 결정
- R/W = 0 : 쓰기 모드로 Input data를 선택된 Word에 저장
- R/W = 1 : 읽기 모드로 선택된 Word에 저장된 값을 output data로 출력

[4 x 4 RAM]



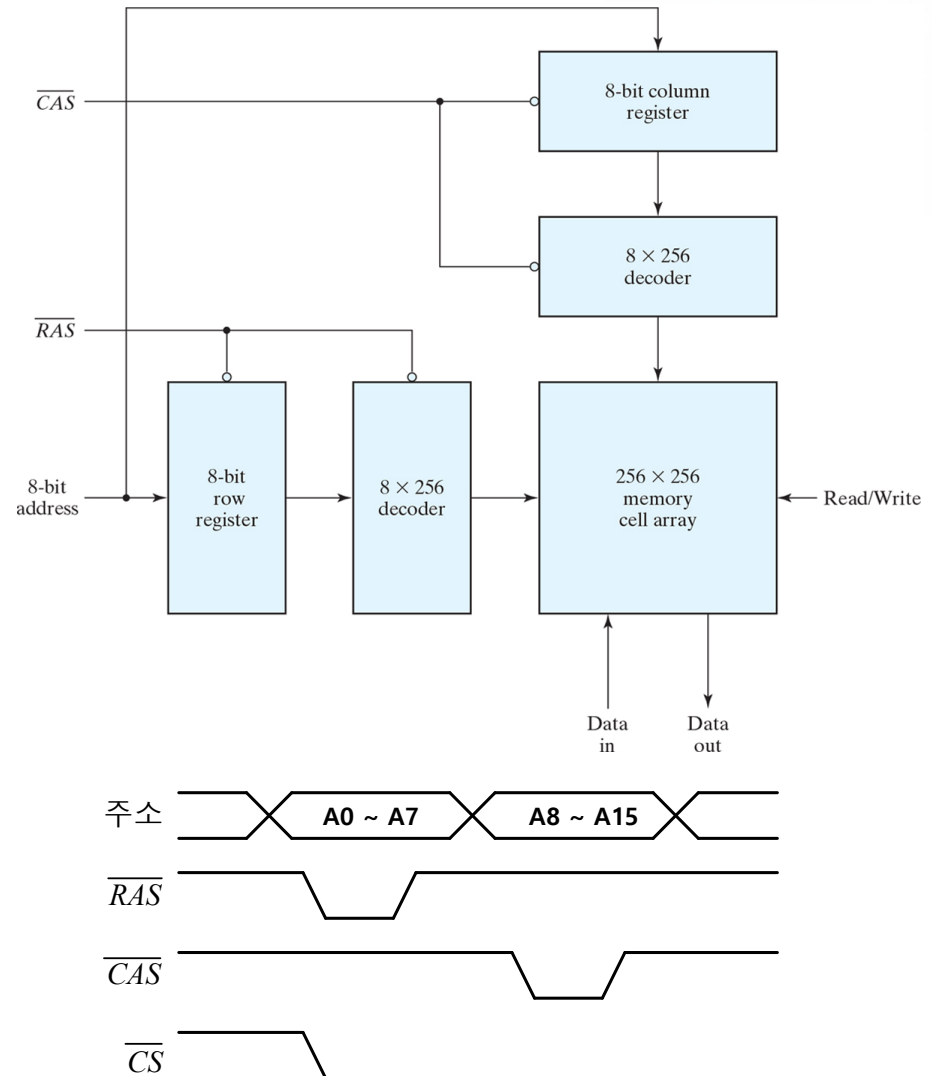
[1K 워드 메모리의 2차원 디코딩 구조]

- 1차원 선택구조를 가진 디코더에 비해 2차원 선택구조를 가진 디코더는 사용되는 게이트 수 및 게이트 입력 수를 줄일 수 있음
- 2차원 디코딩을 위해 메모리 소자의 배열을 가능한 정사각형에 가깝게 배열
- 1K 워드 메모리를 1차원 선택구조를 사용할 경우 10×1024 디코더를 사용하게 되어 1024개의 AND 게이트가 필요하나, 2차원 선택구조를 사용하면 5×32 디코더 두 개를 사용하여 64개의 AND 게이트가 필요



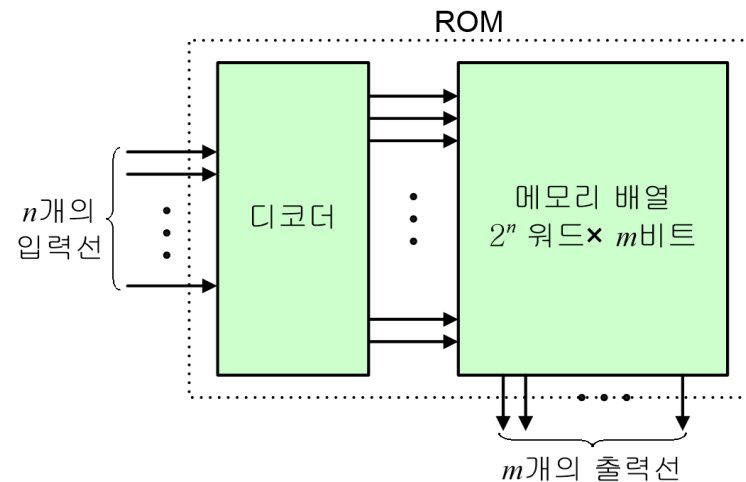
❖ Address Multiplexing

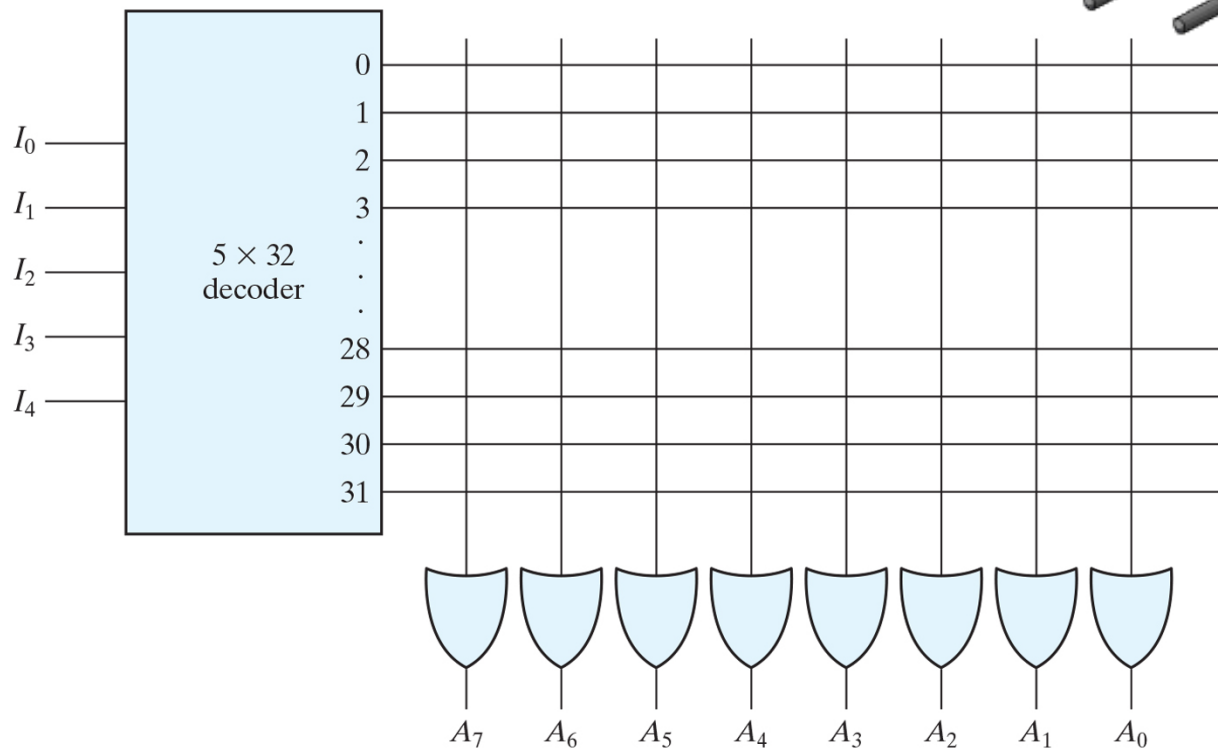
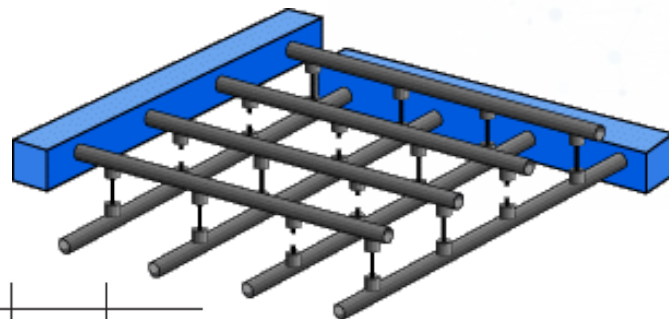
- IC 패키지의 핀 수를 줄이기 위해, 적은 수의 주소(Address) 입력 핀으로 메모리에 접근할 수 있는 Address Multiplexing 기법을 사용
- CAS – Column Address Strobe
- RAS – Row Address Strobe



❖ ROM (Read Only Memory)

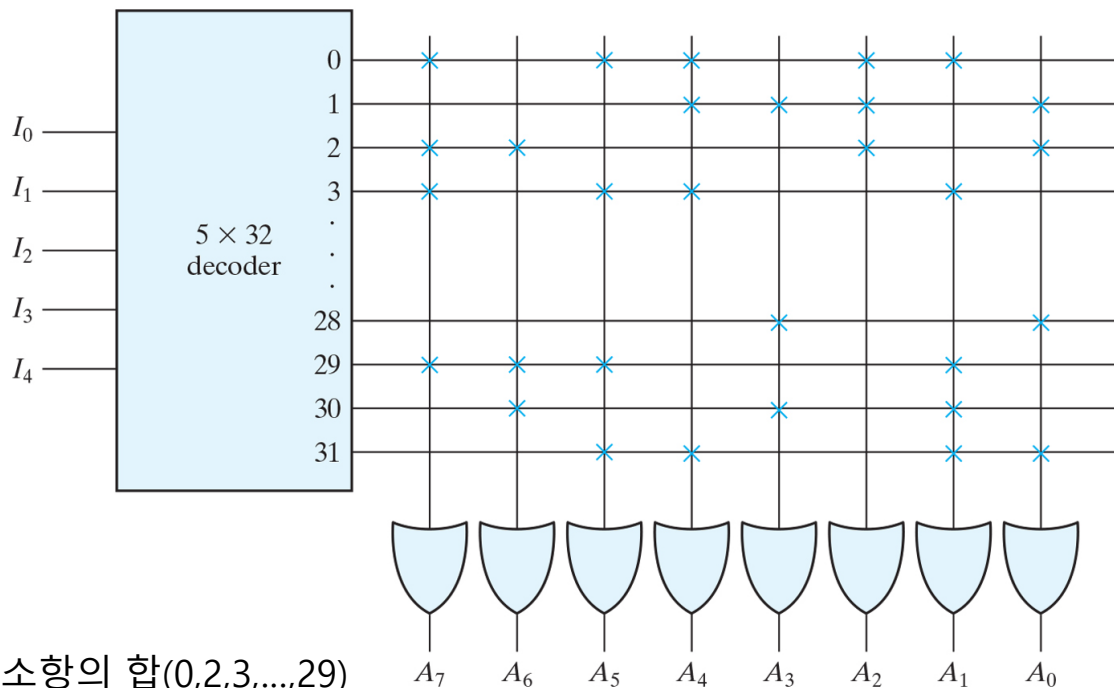
- AND 게이트와 OR 게이트로 구성된 조합논리회로
- AND 게이트는 디코더를 구성
- OR 게이트는 디코더의 출력인 최소항들의 합연산을 위해 사용 됨(SOP논리식)





Inputs					Outputs							
I_4	I_3	I_2	I_1	I_0	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0
0	0	0	0	0	1	0	1	1	0	1	1	0
0	0	0	0	1	0	0	0	1	1	1	0	1
0	0	0	1	0	1	1	0	0	0	1	0	1
0	0	0	1	1	1	0	1	1	0	0	1	0
		$\vdots$							$\vdots$			
1	1	1	0	0	0	0	0	0	1	0	0	1
1	1	1	0	1	1	1	1	0	0	0	1	0
1	1	1	1	0	0	1	0	0	1	0	1	0
1	1	1	1	1	0	0	1	1	0	0	1	1

$I_4 \sim I_0 = 00011$ 일 경우
 $A_7 \sim A_0 = 10110010$

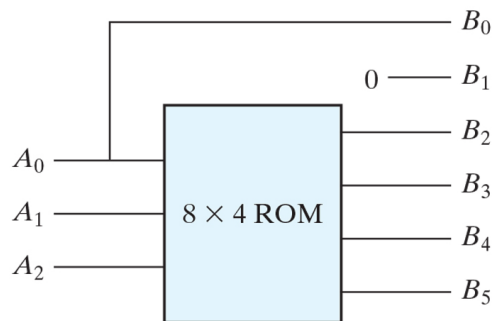


$A_7(I_4, I_3, I_2, I_1, I_0) = \text{최소항의 합}(0, 2, 3, \dots, 29)$

예제 7.1) 다음 조건을 만족하는 ROM 설계

- 3비트 입력
- 입력된 값의 제곱 값 출력

Inputs			Outputs						Decimal
A_2	A_1	A_0	B_5	B_4	B_3	B_2	B_1	B_0	
0	0	0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0	1	1
0	1	0	0	0	0	1	0	0	4
0	1	1	0	0	1	0	0	1	9
1	0	0	0	1	0	0	0	0	16
1	0	1	0	1	1	0	0	1	25
1	1	0	1	0	0	1	0	0	36
1	1	1	1	1	0	0	0	1	49



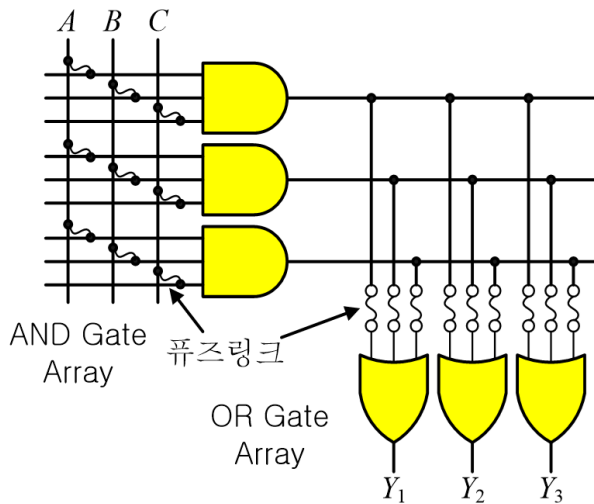
(a) Block diagram

A_2	A_1	A_0	B_5	B_4	B_3	B_2
0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	1	0	0	0	0	1
0	1	1	0	0	1	0
1	0	0	0	1	0	0
1	0	1	0	1	1	0
1	1	0	1	0	0	1
1	1	1	1	1	0	0

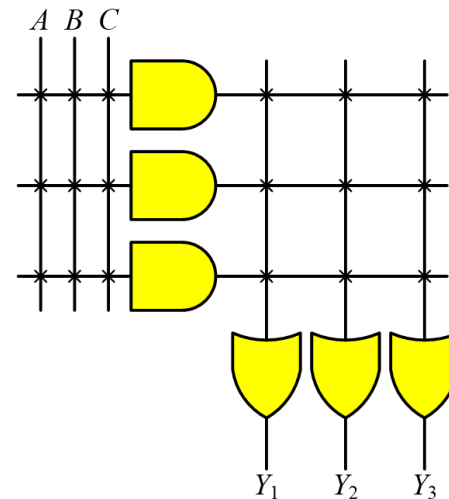
(b) ROM truth table

❖ PLD(Programmable Logic Device)

- 주로 AND 게이트와 OR 게이트의 배열 구조를 가지는 IC를 칭함
- 각 게이트 입력에 퓨즈링크가 연결되어 있어 사용자가 원하는 위치의 퓨즈링크를 전자적으로 Open 상태로 만들어 SOP 형식으로 된 조합논리 함수 구현



[PLD의 퓨즈링크]

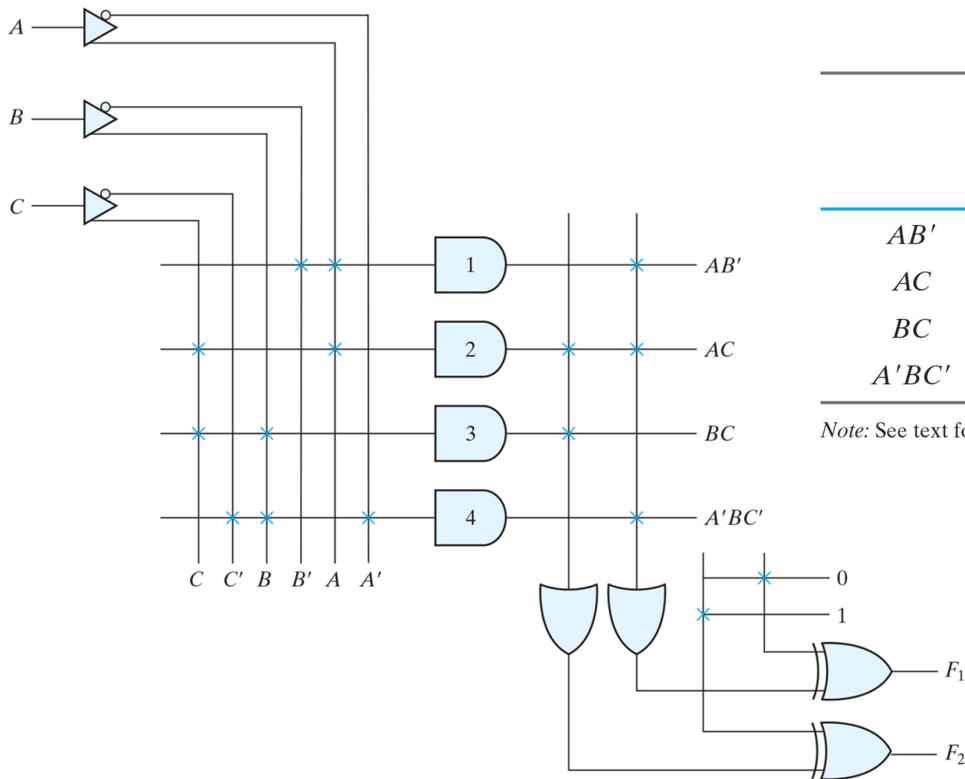


[PLD의 개략도]

- PROM(Programmable ROM) – 디코더의 역할을 하는 고정 AND 배열과 프로그램이 가능한 OR 배열로 구성되어 있으며, 주로 주소 지정 메모리로 사용되며 고정된 AND 게이트의 제약 때문에 논리소자로는 사용하지 않음
- PLA(Programmable Logic Array) – AND 입력과 OR 입력 양쪽을 다 프로그램 할 수 있어서 가장 융통성 있게 프로그램 할 수 있으나, 동작속도와 집적도가 낮음
- PLE(Programmable Logic Element) – AND 입력은 고정되고 OR 입력만을 프로그램 할 수 있는 PLD
- PAL(Programmable Array Logic) – AND 입력만 프로그램 할 수 있고 OR 입력은 고정되어 있으며, 현재 가장 많이 사용되고 있음
- GAL(Generic Array Logic) – 가장 최근에 개발된 소자로 PAL과 마찬가지로 프로그램이 가능한 AND 배열과 고정 OR 배열 및 출력논리로 구성되어 있으나 다시 프로그램 할 수 있고 출력논리도 프로그램이 가능

**PAL / PLA (구형, OTP) → GAL (SPLD, 재기록 가능) →
CPLD (복잡도 증가, 비휘발성) → FPGA (고집적, 고성능, 주력)**

- PLA(Programmable Logic Array) – AND 입력과 OR 입력 양쪽을 다 프로그램 할 수 있어서 가장 융통성 있게 프로그램 할 수 있으나, 동작속도와 집적도가 낮음
- 출력단의 XOR 게이트를 이용해 보수처리 가능
- IC 패키지의 제한된 AND 게이트만 가지고 있으므로 곱의 항의 수를 줄이는 것이 중요



		Inputs			Outputs (T) (C)	
	Product Term	A	B	C	F_1	F_2
	AB'	1	0	—	1	—
	AC	1	—	1	1	1
	BC	—	1	1	—	1
	$A'BC'$	0	1	0	1	—

Note: See text for meanings of dashes.

예제 7.2) 다음 조건을 만족하는 PLA 구현

$$F_1(A, B, C) = \sum (0, 1, 2, 4)$$

$$F_2(A, B, C) = \sum (0, 5, 6, 7)$$

		<i>BC</i>		<i>B</i>	
		00	01	11	10
<i>A</i>	0	1	1	0	1
	1	1	0	0	0

C

$$F_1 = A'B' + A'C' + B'C'$$

$$F_1 = (AB + AC + BC)'$$

		<i>BC</i>		<i>B</i>	
		00	01	11	10
<i>A</i>	0	1	0	0	0
	1	0	1	1	1

C

$$F_2 = AB + AC + A'B'C'$$

$$F_2 = (A'C + A'B + AB'C')'$$

PLA programming table

		Inputs			Outputs	
		Product term			(C)	(T)
			<i>A</i>	<i>B</i>	<i>C</i>	
			<i>F</i> ₁	<i>F</i> ₂		
<i>AB</i>	1	1	1	–	1	1
<i>AC</i>	2	1	–	1	1	1
<i>BC</i>	3	–	1	1	1	–
<i>A'B'C'</i>	4	0	0	0	–	1

- PAL(Programmable Array Logic) – AND 입력만 프로그램 할 수 있고 OR 입력은 고정
- AND 게이트만으로 프로그래밍 하기 때문에 PLA에 비해 제약이 따름

$$w(A,B,C,D) = \sum(2,12,13)$$

$$x(A,B,C,D) = \sum(7,8,9,10,11,12,14,15)$$

$$y(A,B,C,D) = \sum(0,2,3,4,5,6,7,8,10,11,15)$$

$$z(A,B,C,D) = \sum(1,2,8,12,13)$$

$$w = ABC' + A'B'CD'$$

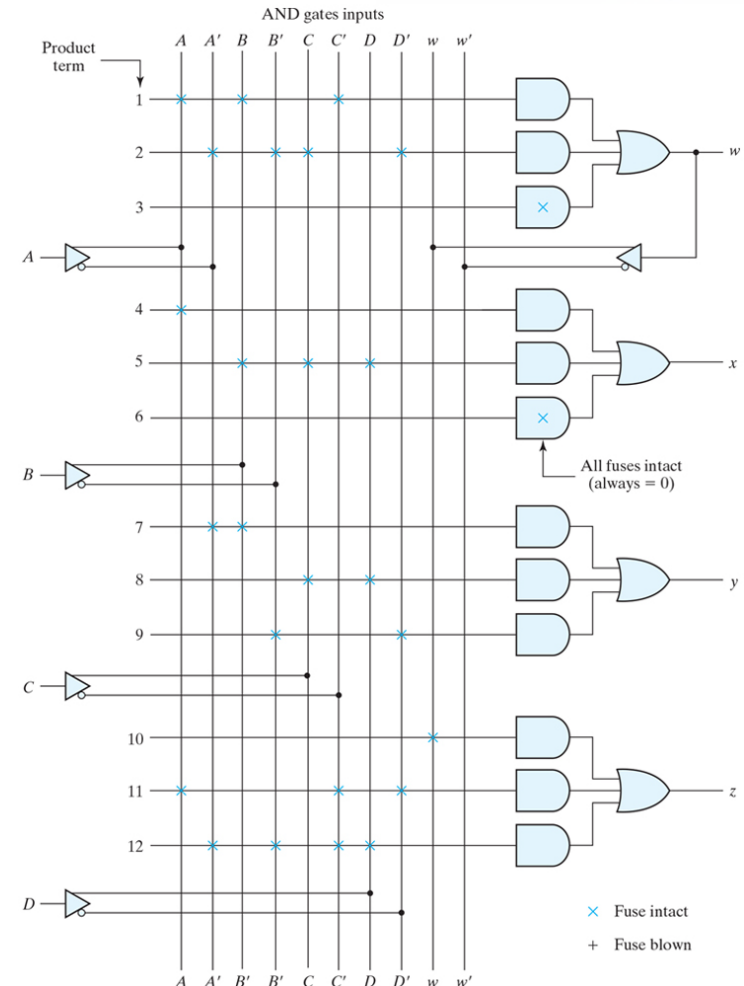
$$x = A + BCD$$

$$y = A'B + CD + B'D'$$

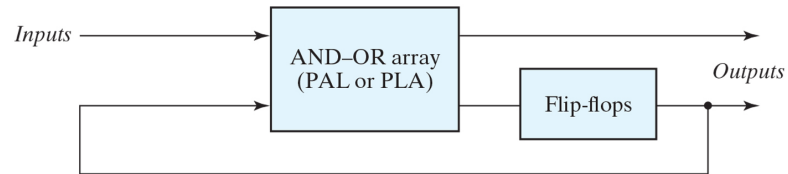
$$z = ABC' + A'B'CD' + AC'D' + A'B'C'D$$

$$= w + AC'D' + A'B'C'D$$

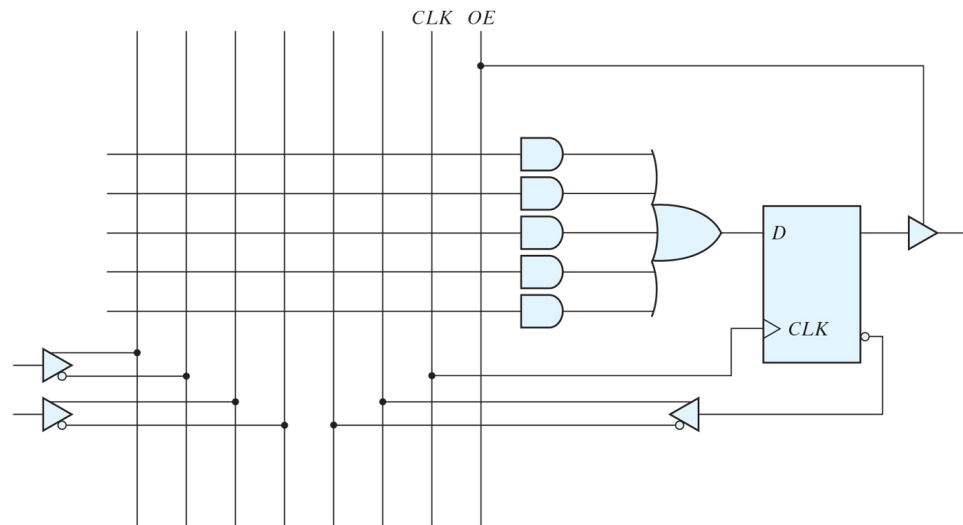
Product Term	AND Inputs					Outputs
	A	B	C	D	w	
1	1	1	0	—	—	$w = ABC' + A'B'CD'$
2	0	0	1	0	—	
3	—	—	—	—	—	
4	1	—	—	—	—	$x = A + BCD$
5	—	1	1	1	—	
6	—	—	—	—	—	
7	0	1	—	—	—	$y = A'B + CD + B'D'$
8	—	—	1	1	—	
9	—	0	—	0	—	
10	—	—	—	—	1	$z = w + AC'D' + A'B'C'D$
11	1	—	0	0	—	
12	0	0	0	1	—	



❖ 순차적 프로그래머블 논리 소자

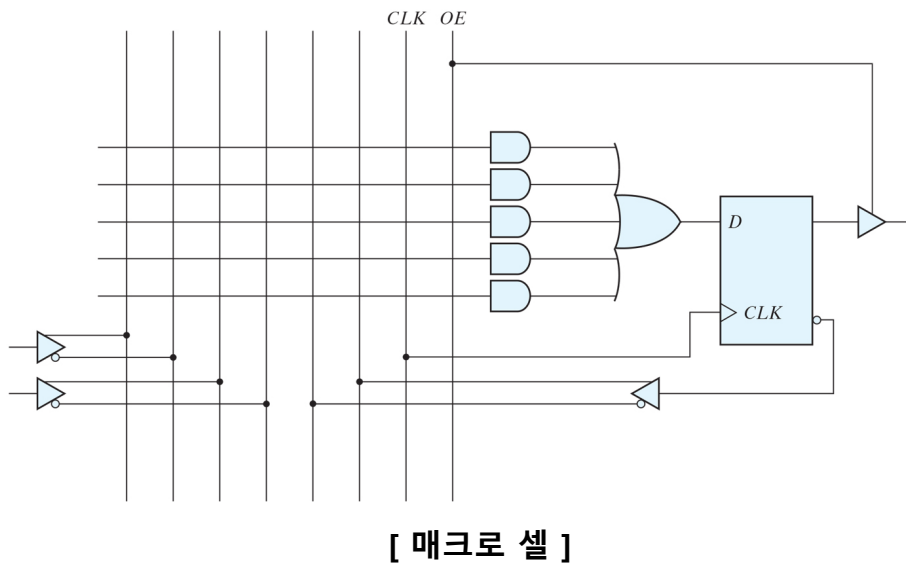


- 조합 PLD는 게이트들 로만 구성되어 있어 플립플롭이 필요한 경우가 있음
- 순차적 프로그래머블 소자는 게이트와 플립플롭을 모두 포함하는 소자를 칭함
- 내부 구조는 복잡한 관계로 종류에 대해서만 언급



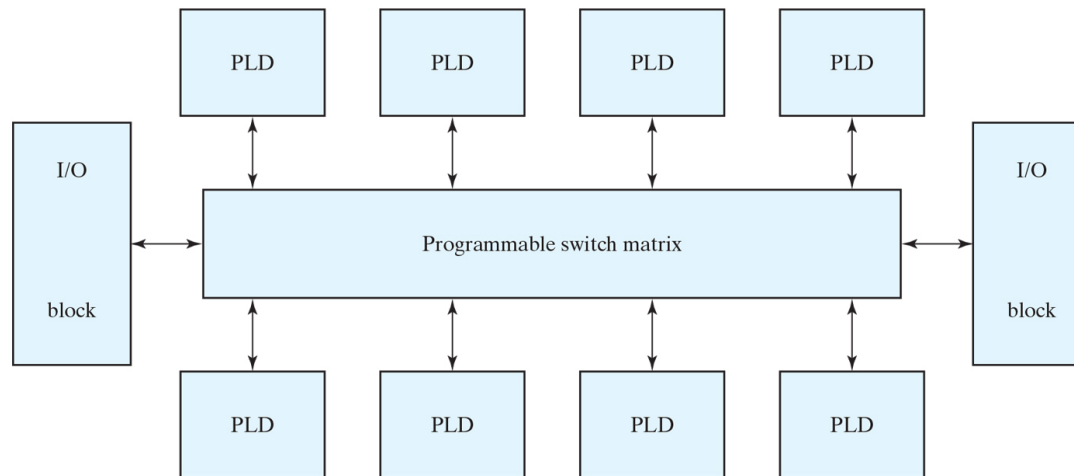
❖ GAL - SPLD (Sequential Programmable Logic Device)

- PAL + D Flip-Flop
- 위와 같은 조합을 기본단위(매크로 셀)로 패키지 구성



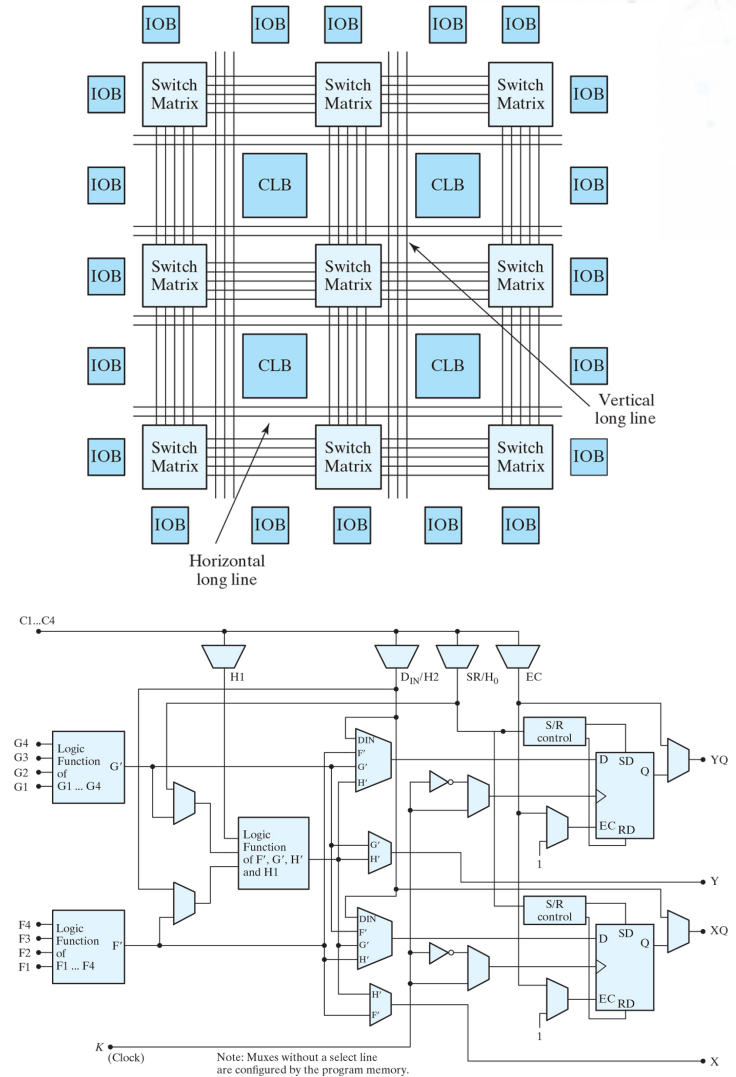
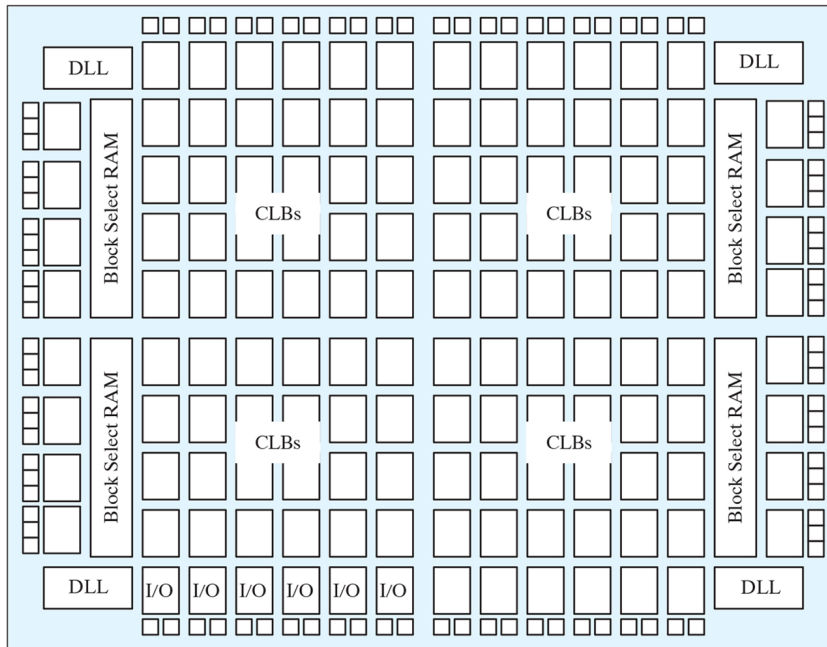
❖ CPLD (Complex Programmable Logic Device)

- 각각의 PLD 모듈을 프로그래밍이 가능한 스위칭 매트릭스를 통해 구조를 변경하여 기능 구현
- 각 PLD 역시 매크로 셀로 구성되어 있음



❖ FPGA (Field Programmable Gate Array)

- 사용자가 직접 프로그래밍 할 수 있는 VLSI 회로
- 룩업테이블, 멀티플렉서, 게이트, 플립플롭, RAM 등으로 구성



메모리 및 프로그래머블 논리

